

ECE 5242 Project 4: Final Submission

Hugo Onghai¹

¹ECE Department at Cornell Tech

Abstract

This paper presents implementations of model-based and model-free reinforcement learning algorithms: policy iteration, Q-learning, and REINFORCE. They are applied to maze-routing problems and physics-informed games provided by OpenAI Gym. It is found that all methods can successfully be made converge to a non-trivial optimized policy, Q-table, etc. though the specific policy reached is determined by many hyperparameters. We provide learning curves obtained with different hyperparameters. We find that policy iteration yields an optimal Q-table that typically collects two of the three flags in the maze, and artificially increasing the discount factor promotes collecting all three. Q-learning converged more closely to that optimal Q-table with larger learning rates and greedy-epsilon values for the same maze problem. REINFORCE was more difficult to converge on the more complex gym environments but with a near-unity discount factor and a small learning rate it could for Acrobot, with high variance but without collapsing the policy. A Mountain Car agent could not be taught to consistently reach the goal state, yet. Future work would go into applying Q-learning on a discretized state-space or applying more in-depth hyperparameter tuning on REINFORCE.

Keywords: Reinforcement Learning, Model-Based, Model-Free, Policy Gradient

Correspondence: hpo8@cornell.edu

1. Introduction

Reinforcement learning is learning how to act in an environment to maximize a reward. Actions are reinforced if they (or subsequent actions) yield reward, and in turn, unfavorable actions are ignored if they do not yield reward. This is a fundamental way with which intelligent autonomous systems, called agents, learn to navigate environments and extract reward.

Agent navigation through an environment is often modelled as a Markov decision process. One arbitrary episode might look something like this:

$$(1) \quad (S_0, A_0, R_1) \rightarrow (S_1, A_1, R_2) \rightarrow \dots \rightarrow (S_{T-1}, A_{T-1}, R_T)$$

An agent in state S_0 takes an action A_0 and receives reward R_1 . This action brings them to S_1 , where they take another action and receive another reward. This repeats until either a maximum number of actions are taken or a goal state is reached.

A policy (often denoted as π) informs an agent on how to take actions at each state, and each of the algorithms discussed hereafter optimize their policy differently.

2. Problem Statement

Given an environment and desired goal state, how can we enable an agent to learn how to act to reach that goal state? With known algorithms to do this, how can we then tune our hyperparameters to extract better performance or convergence?

3. Technical Approach

We implement three well-known algorithms in reinforcement learning: policy iteration, Q-learning, and REINFORCE. We apply policy iteration and Q-learning to

3.1. Policy Iteration

Policy iteration is a model-based method that develops a deterministic policy $\pi(s)$ through successive rounds of policy evaluation and policy improvement.

In policy evaluation, each state's value is derived from the value of its subsequent states and the probability of transitioning there and the reward gained in doing so. In this fashion, all states are valued recursively from the goal state, whose non-zero reward "trickles down" through the rest of the Markov decision process. This recursive procedure to construct a value function is often referred to as dynamic programming.

The probability of transition from one state in the maze to another was complicated by a "slipping probability" in which the agent would have a 10% chance of unintendedly moving 90° clockwise. These transition probabilities are generated in `src/project4/mdp.py` and slipping in an episode is implemented for us in the provided `maze.py` located at `src/project4/maze.py`.

In policy improvement, the new policy is constructed. The better policy uses the new value function to instruct the agent to take the action that brings them to the highest-valued states (or future states, assuming a non-zero discount factor).

Note that this is different from value iteration, which we also could have implemented. Value iteration bypasses a policy by refining a value function that the agent can follow maximally.

3.2. Q-learning

Unlike policy iteration, which requires us to map out the full Markov decision process, Q-learning does not need such a model ahead of time. This is why Q-learning is a model-free method and policy iteration is a model-based method. In Q-learning, a so-called Q-table enumerating all possible states and actions is constructed additively.

To get an understanding of the environment, Q-learning stochastically traverses the environment, exploring new reward paths and exploiting known good ones. The rate at which actions are explorative or exploitative is governed by the ε -greedy policy. In our implementation, this is merely a Bernoulli random variable governed by constant $\varepsilon \in [0, 1]$. Upon measurement, the agent acts to explore with a probability of ε . At each (S_t, A_t, R_{t+1}) for any $0 \leq t < T$, the state-action pair (S_t, A_t) is updated based on the reward it yielded (R_{t+1}) and the Q-table value of the next state-action pair in the episode (S_{t+1}, A_{t+1}) .

Over many episodes, the state-action pairs converge to their optimal Q-table values, in expectation. This fact is defined by the Bellman equation:

$$(2) \quad Q^*(s, a) = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]$$

Then, after convergence, the optimal policy is implicitly derived from the Q-table. We takes only greedy steps:

$$(3) \quad a^* = \underset{a}{\operatorname{argmax}} Q(s, a) \text{ for any } s \in S$$

3.3. REINFORCE

REINFORCE is unlike policy iteration and Q-learning in the sense that it defines a probabilistic policy, which is updated with a stochastic gradient ascent step. Similar to Q-learning it is a model-free method.

In REINFORCE, we iterative update a differentiable policy parametrization based on a set of acted episodes. While the policy parameterization can be formulated in many ways, our parametrization $\theta \in \mathbb{R}^{\text{num_actions} \times \text{state_dim}}$ is simply a linear function of the state variables. The preference h of a state $s \in \mathbb{R}^{\text{state_dim}}$ is then calculated as

$$(4) \quad h(s, \theta) = \theta^\top s$$

and our policy takes the exponential soft-max to convert these preferences into a probabilistic policy

$$(5) \quad \pi(a|s, \theta) := \frac{\exp(h(s, \theta))}{\sum_b \exp(h(s, \theta))}$$

which gives the best actions the highest probability of being chosen.

Since π is differentiable, we can take its gradient with respect to θ , which informs us which direction in θ -space we should update our parameterization to increase the probability of actions that yield high returns:

$$(6) \quad \theta \leftarrow \theta + \alpha \gamma^t G_t \nabla \ln \pi(A_t | S_t, \theta)$$

However, different from this canonical version, I introduced a baseline (as was done recommended in the project spec and explained in by Sutton and Barto). My baseline was a simple moving average of total rewards, which was subtracted from the future return G_t . Therefore, my REINFORCE update looks more like this:

$$(7) \quad \theta \leftarrow \theta + \alpha * \gamma^t \delta \nabla \ln \pi(A_t | S_t, \theta), \text{ where } \delta = G_t - \frac{1}{t} \left(\sum_0^t G_t \right)$$

Because REINFORCE uses a policy-gradient, it is prone to very noisy convergence and policy collapse. Policy collapse especially occurs when the learning rate α is too large, and the optimal θ is “overshot” at each step. We show an example of policy collapse later in our results.

4. Results and Discussion

In this section, we discuss the performance of our three algorithms in training agents in their given environment. We also discuss how hyperparameters affected this performance.

4.1. Policy Iteration in the Maze Environment

[To view an video of \$Q^*\$ -informed optimal path \(with slipping\), click here!](#)

Policy iteration successful learned to reach the goal state and pick up 2 flags along the way.

Although we were instructed to keep the discount factor $\gamma = 0.9$, we ran some heuristic tests with different discount factors to explore other discount factors. We found that $\gamma = 0.99$ had a very difficult time of reaching the goal state. After many evaluations of the Q^* -informed policy, the agent would get stuck in corners or at flags. The reason for this is uncertain, however, I presume that having such a large discount factor may “confuse” the agent in trying to extend its episode path to get more reward, which leads to worse policy stabilization. Or, a discount factor this large is not appropriate for a discrete environment this small (only 112 states). In contrast, with $\gamma = 0.8$, the agent successfully reach the goal state with a reward of 1.0. The agent would collect only the top flag which offered the least deviation from the shortest to-goal path.

More tuning could be done to try and get the agent to learn to grab all 3 flags, but presumably its difficult without a more complex reward structure. Since reward is only given at the goal state, it seems that the agent puts significant emphasis on reaching the goal state in as few of moves as possible. Perhaps if there was some reward for getting the farthest flags, the agent would go for them more often (or even with a smaller discount factor).

4.2. Q-Learning in the Maze Environment

Like policy iteration with $\gamma = 0.9$, Q-learning also converged to a two-flag optimal reward with the best hyperparameter tuning. [Figure 1](#) shows Q-learning convergence with many sets of parameters, where convergence is treated as the RMSE of the Q-table with respect to Q^* found with policy iteration ($\gamma = 0.9$).

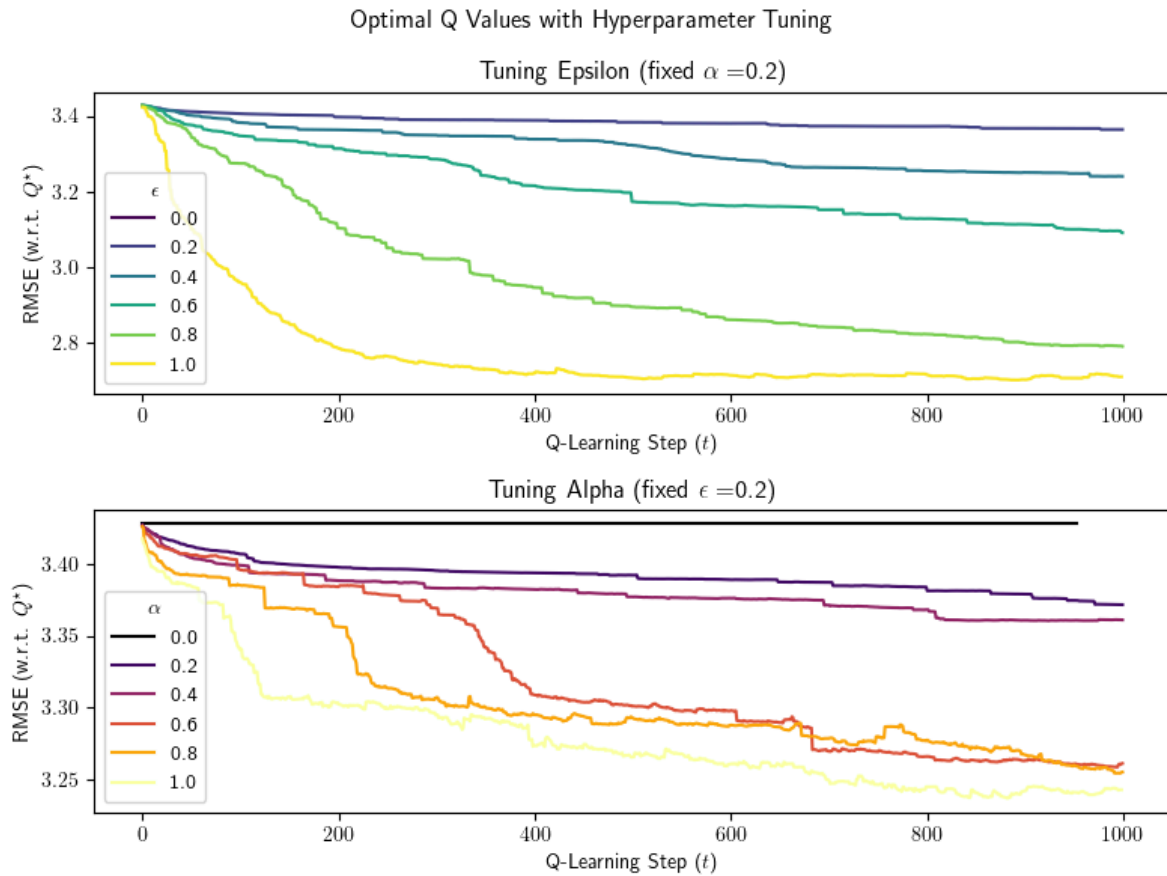


Figure 1 – Hyperparameter sweeps, with either learning rate α (top) or ϵ (bottom) fixed. While RMSE with respect to Q^* from policy iteration decreased the most and fastest with a larger learning rate α and ϵ .

It was found that increasing learning rate α and ϵ led to faster convergence. To compare these convergence rates and the converged average reward, we also performed performance evaluation on several combinations of α and ϵ , as shown in [Figure 2](#).

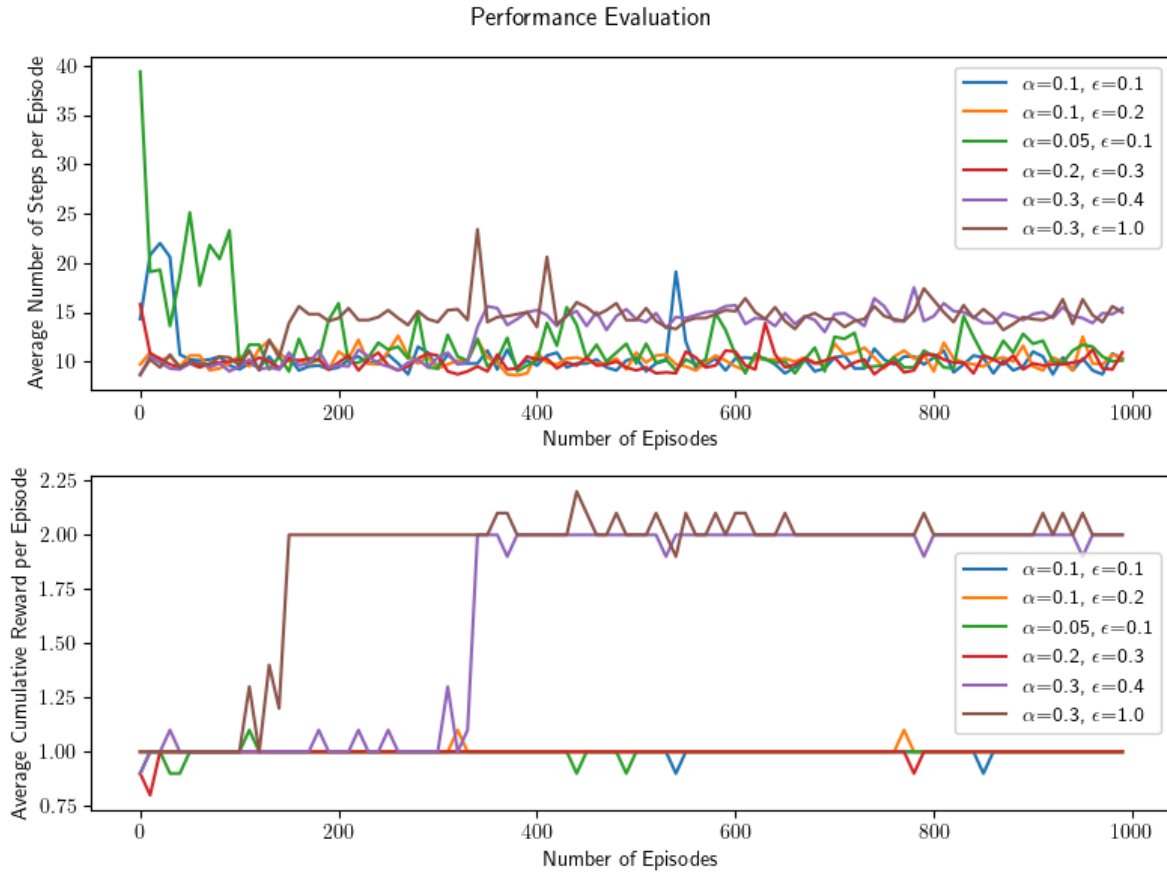


Figure 2 – The average number of steps per episode (top) and average cumulative reward G_0 per episode (bottom). For convenient comparison, I plotted Laura’s hyperparameter combinations and one of my combinations with $\epsilon = 1.0$ which was found to be good in the parameter sweep.

Roughly speaking, we can see from the the average number of steps that ($\alpha = 0.05, \epsilon = 0.1$) reduced its step count slowly. Moreover, we can also see how the simulations that converge onto an average cumulative reward of 2.0 appear to converge to a higher number of steps per episode, which makes sense because it takes more steps to get more flags.

From the bottom plot, we can see that ($\alpha = 0.3, \epsilon = 1.0$) converges to an average reward of 2.0 before ($\alpha = 0.3, \epsilon = 0.4$) despite both having the same learning rate α . This suggests that most of your time early on is best spent exploring. This agrees with what we discussed in lecture and with more advanced ϵ -greedy methods, like the adaptive ϵ .

4.3. REINFORCE in the Acrobot and MountainCar Environments

After implementing REINFORCE, it was applied to two OpenAI Gymnasium environments: `acrobot-v1` and `mountaincar-v0`.

4.3.1. Acrobot.

For Acrobot, the following parameters were used.

Table 1 – Hyperparameters used with REINFORCE for Acrobot

γ Discount Factor	0.99
α Learning Rate	0.00001
No. Episodes	5000
Max. Steps per Episode	1000

During training, I used Gymnasium’s video recording functionality to show the progress of the agent every 1000 episodes.

[To view videos of Acrobot training with REINFORCE, click here!](#)

In this environment, every step that does not result in reaching the goal height (shown in the videos with the black line) receives a reward of -1 . A step that does reach the goal receives a rewards of 0. This way, agents are incentivized to reach the goal and in a few steps as possible. We can see how reducing the number of steps to reach the goal significantly increases a robot’s total reward over an episode G_0 in Figure 3.

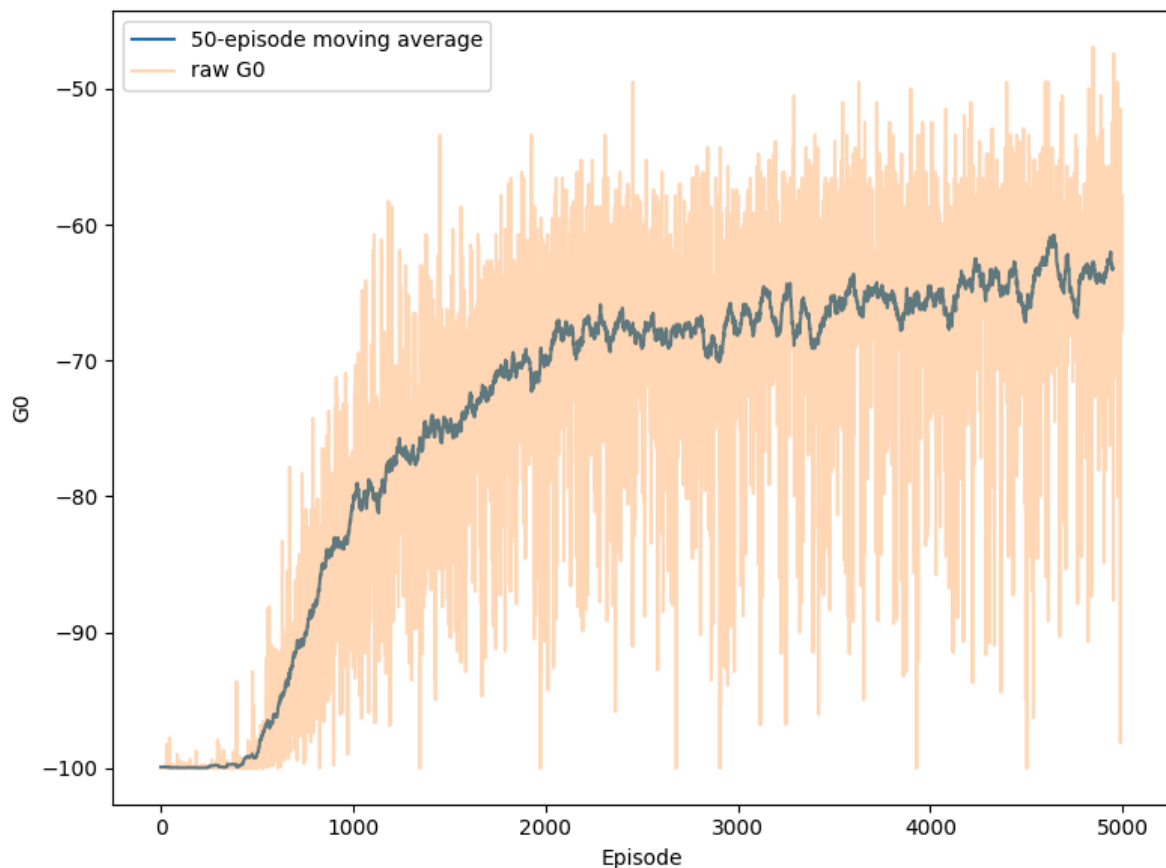


Figure 3 – The total reward over an episode during training of Acrobot with REINFORCE. A clear converging trend in the 50-episode moving average is noticeable, despite the noise in the raw G_0 .

I did not do any normalization of the state-space for this, which likely means my θ -space was not well-conditioned for efficient stochastic gradient ascent. This is why I believe I needed to make my learning rate so incredibly small.

I had to have a large discount factor since it typically took many steps, especially in early episodes, to reach the goal state. Even after all 5000 steps, it still took around 90 to 100 steps to reach the goal state. Having a very large discount factor allows the REINFORCE algorithm to “look further into the future” when training the agent’s optimal policy.

4.3.2. Mountain Car.

For Mountain Car, the following parameters were used.

Table 2 – Hyperparameters used with REINFORCE for Mountain Car

γ Discount Factor	$1 - 10^{-4}$
α Learning Rate	0.001
No. Episodes	5000
Max. Steps per Episode	1000

In this environment, similar to Acrobot, every step that does not result in reaching the flag on the top of the hill is given a reward of -1 , else 0 .

Unfortunately, I was not able to get it to consistently reach the top of the hill. Miraculously, the car managed to get lucky and reach the hill on the first episode, but it never could again after that. I recorded it and the video can be found [here](#).

I’m not sure how this happened, but it was right after I added clustering to my action sampling in time. My guess is my initialization of θ was such that this perfectly happened by accident. As you can see in the video, the car first accelerated all the way left, and then uncontrollably reached the right.

The rest of the trials were not so successful. Their videos can be found [here](#). As such, my performance evaluation plot looks quite silly.

[For the full folder of Mountain Car videos, click here!](#)

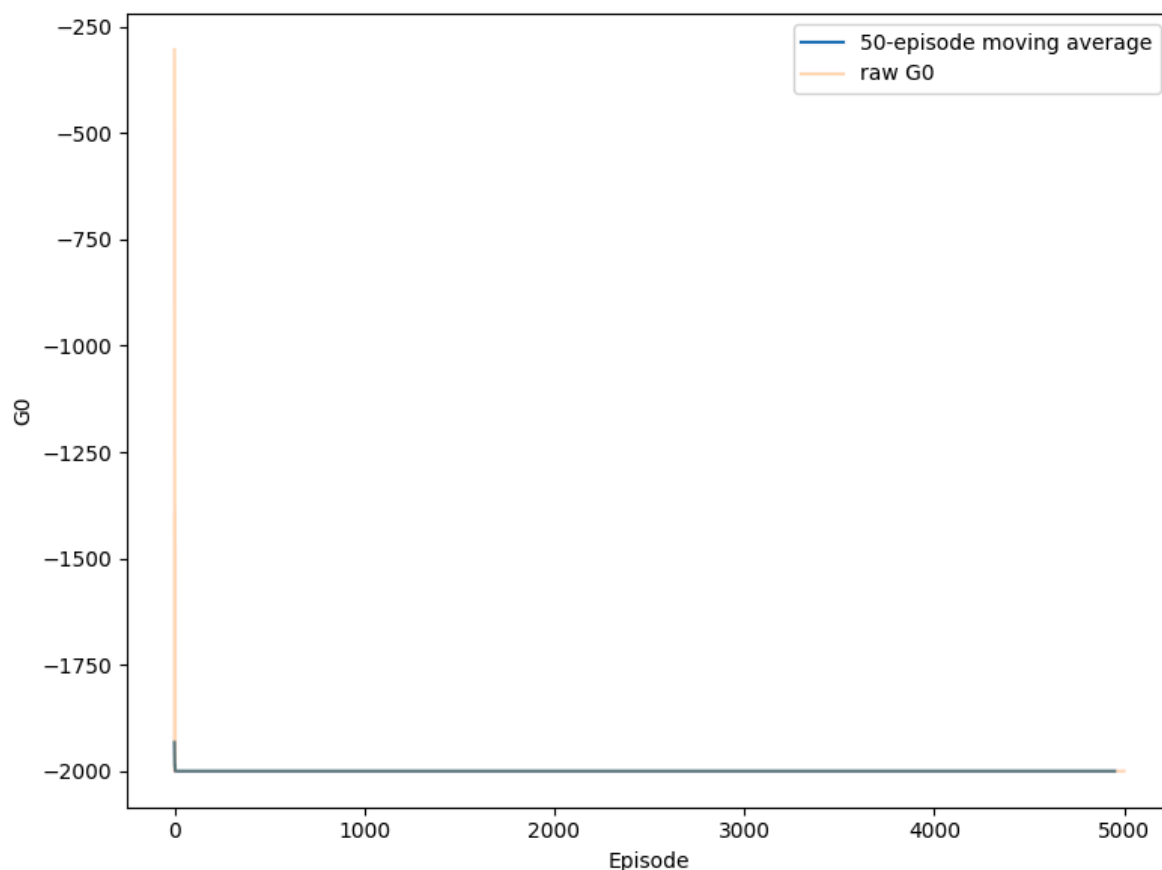


Figure 4 – The performance evaluation for my strange mountain car. As you can see, the first episode outperformed all of the rest.

Due to time constraints, I did not have an opportunity to work on implementing a Q-learning for these projects or to discretize the sample space in order to do that.

Beyond this, I would also have liked to fine-tune my REINFORCE hyperparameters more. I think that my Mountain Car may have had too low of a learning rate or too high of a discount factor, although lower discount factors did not seem successful either. Cluster more steps in a row could have also been better, since my intuition tells me that the car should always accelerate right during times when its velocity is directed rightwards. Clustering could help simplify our car to be more likely to do this.

5. Concluding Remarks

In conclusion, reinforcement learning offers a variety of effective strategies to teach an agent how to act in an environment to obtain a specific reward. However, much consideration must be put into how that reward is defined, how the environment is defined, and even what state that agent is in within that environment. We discussed how these considerations can be formatted into a Markov decision process. In model-based methods, we can build models for how an agent might transition through the states and build a optimal policy for extracting future reward. In model-free methods, we can collect information through episodic trials, learning about the environment and what earns our agent reward as it goes along. Model-free methods are

certainly more difficult to converge, however, in very complex environments with large state-spaces, they can also prove more versatile (though it depends, since a giant Q-table can be memory-inefficient too).

6. Sources

- (1) Professor Lee's Reinforcement Learning Lectures for our Intelligent Autonomous Systems course (ECE 5242).
- (2) "Reinforcement Learning: An Introduction, Second Edition" by Sutton and Barto.
- (3) Gymnasium Documentation: <https://gymnasium.farama.org/>

7. Appendix

- All videos can be found [here](#).
- My GitHub repository for this project can be found [here](#).